Query Data with Ecto.Query

Ecto provides robust query functions—you can write most queries without writing SQL by hand. And if you run into a query that can't be written with the Ecto query language, you can safely use handwritten SQL.

We're going to build our data layer—everything needed to query and persist data in our application. The functions and patterns we use will be explained along the way. But first, we need to go over a module that we're going to use many times in this section.

The Repo Module

The generator that we used to create our application included an important (and simple) module. The `PhoneApp.Repo` module is how we'll handle all interactions with the database. The module is only one line of code:

phone_app/lib/phone_app/repo.ex

```
defmodule PhoneApp.Repo do
  use Ecto.Repo,
    otp_app: :phone_app,
    adapter: Ecto.Adapters.Postgres
end
```

The single line of code in this module—`use Ecto.Repo`—defines a bunch of functions. It includes functions to insert data, update data, make queries, execute raw SQL statements, and more. These functions could be customized by overriding them, but that's an advanced technique that you won't commonly need.

The Repo module is a core part of Ecto's design. Every single query in our application goes through our Repo. A Repo connects to a single database, so most applications have a single Repo module, but there's nothing stopping us from having multiple Repos that connect to different databases.

Let's use the `PhoneApp.Repo` module to make some queries!

Query and Persist Contacts

Our application will interact with `Schema.Contact` in a simple way. We only need to be able to retrieve a contact by its ID and create a new contact using a unique phone number.

There are many different ways we could organize the queries for our application. The approach used in this book is by no means the definitive way, but it's something that has served me well in small and scaled applications. To